

994. Rotting Oranges Medium

Acceptance: 57.9% | [LeetCode](#)

You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:

```
**Input:** grid = [[2,1,1],[1,1,0],[0,1,1]]  
**Output:** 4
```

Example 2:

```
**Input:** grid = [[2,1,1],[0,1,1],[1,0,1]]  
**Output:** -1  
**Explanation:** The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.
```

Example 3:

```
**Input:** grid = [[0,2]]  
**Output:** 0  
**Explanation:** Since there are already no fresh oranges at minute 0, the answer is just 0.
```

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ is 0, 1, or 2.

Claude Opus Solution

Rotting Oranges - Solution

1. Intuition

The key insight is recognizing this as a **multi-source BFS** problem.

- Rotting spreads **simultaneously** from ALL rotten oranges, one layer (minute) at a time.
- This "spreading in waves" behavior is exactly what BFS models: each BFS level corresponds to one minute.
- The minimum minutes equals the **number of BFS levels** needed to reach all fresh oranges.
- If some fresh orange is unreachable (isolated by empty cells), we return -1.

The reference solution uses a clever trick: instead of running BFS from a single source, we **seed the queue with ALL rotten oranges at the start**. This way they all expand outward at the same rate, perfectly modeling simultaneous rotting.

2. Approach (Pseudocode)

1. Scan the entire grid:
 - Count all FRESH oranges (1)
 - Add all ROTTEN oranges (2) to a queue ← multi-source start
2. If there are no fresh oranges → return 0 (nothing to rot)
3. BFS level-by-level (each level = 1 minute):
 - Process every orange currently in the queue (this minute's rotten set)
 - For each, check 4 neighbors:
 - If neighbor is fresh → rot it, decrement fresh count, enqueue
 - After processing a full level, increment the minute counter
4. Stop when no fresh oranges remain OR queue is empty.
5. If fresh count == 0 → return minutes; else → return -1

3. Python Solution

```
from collections import deque  
from typing import List
```

```

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        queue = deque()      # holds positions of CURRENT rotten oranges
        fresh = 0            # count of fresh oranges remaining

        # --- Step 1: Initialize ---
        # Find all rotten oranges (BFS sources) and count fresh ones.
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 2:
                    queue.append((r, c))    # multi-source: seed all rotten
                elif grid[r][c] == 1:
                    fresh += 1

        # The 4-directional movement vectors (up, down, left, right)
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        minutes = 0

        # --- Step 2: Multi-source BFS ---
        # Continue only while there are rotten oranges AND fresh ones to rot.
        # The 'fresh' check prevents counting an extra unnecessary minute.
        while queue and fresh > 0:
            # Process exactly one level (= one minute's worth of rotting).
            # We snapshot the current level size so newly added oranges
            # are processed in the NEXT iteration (next minute).
            for _ in range(len(queue)):
                r, c = queue.popleft()

                # Try to rot each of the 4 neighbors
                for dr, dc in directions:
                    nr, nc = r + dr, c + dc
                    # Check bounds AND that the neighbor is fresh
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                        grid[nr][nc] = 2    # rot the fresh orange
                        fresh -= 1           # one less fresh orange
                        queue.append((nr, nc)) # it becomes a new source

            minutes += 1 # one full level processed = one minute elapsed

        # --- Step 3: Final check ---
        # If any fresh orange remains, it was unreachable -> impossible.
        return minutes if fresh == 0 else -1

```

Why the `fresh > 0` guard matters

Without checking `fresh > 0` in the while condition, we might process one extra empty level and over-count `minutes`. By stopping as soon as `fresh == 0`, the counter stays accurate. (This also elegantly handles **Example 3**: if `fresh == 0` initially, the loop never runs and we return `0`.)

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(m \times n)$	Initial scan visits every cell once. In BFS, each cell is enqueued/dequeued at most once (we mark it rotten when visited), so total work is bounded by the grid size.
Space	$O(m \times n)$	In the worst case (every cell is rotten initially, or all become rotten), the queue can hold up to $m \times n$ positions.

5. Alternative Approaches

A) BFS without modifying the grid (using a `visited` set) Instead of mutating `grid` to mark rotting, track visited cells in a set. Cleaner if you must preserve input, but uses extra memory.

B) Iterative simulation (no queue) Repeatedly sweep the entire grid; each pass, rot all fresh oranges adjacent to a rotten one. Repeat until no changes occur. - Time: $O((m \times n)^2)$ in the worst case — much slower because each pass costs $O(m \times n)$ and there can be $O(m \times n)$ passes. **Not recommended.**

C) DFS with timestamps Run DFS from each rotten source, recording the *earliest* minute each cell becomes rotten (overwriting only if a smaller time is found). The answer is the max timestamp. This works but is trickier and offers no efficiency gain over BFS. BFS is the natural fit because it inherently processes cells in increasing distance (= minute) order.

Verdict: Multi-source BFS is the optimal and most intuitive solution — it directly mirrors the simultaneous, wave-like rotting process described in the problem.

Codex Solution

1. Intuition

This problem is a classic **multi-source Breadth-First Search BFS** problem.

Each rotten orange spreads rot to adjacent fresh oranges every minute. Since all rotten oranges spread **simultaneously**, we should start BFS from **all initially rotten oranges at the same time**.

Think of each rotten orange as a BFS source. Each BFS level represents one minute passing.

Key idea:

- Initial rotten oranges are minute `0`.
- Their fresh neighbors rot at minute `1`.
- Those oranges rot their neighbors at minute `2`.
- And so on.

If some fresh orange is never reached by BFS, then it can never rot, so return `-1`.

2. Approach

Pseudocode

```
function orangesRotting(grid):
    rows = number of rows
    cols = number of columns

    queue = empty queue
    fresh = 0

    for each cell in grid:
        if cell is fresh orange:
            fresh += 1
        else if cell is rotten orange:
            add cell to queue

    minutes = 0

    directions = up, down, left, right

    while queue is not empty and fresh > 0:
        process all oranges currently in queue
        this represents one minute

        for each rotten orange in current level:
            for each 4-directional neighbor:
                if neighbor is fresh:
                    make it rotten
                    fresh -= 1
                    add neighbor to queue

        minutes += 1

    if fresh == 0:
        return minutes
    else:
        return -1
```

3. Python Solution

```
from collections import deque
from typing import List

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        rows = len(grid)
        cols = len(grid[0])

        # Queue stores the positions of rotten oranges.
        # We start with all initially rotten oranges.
        queue = deque()

        # Count how many fresh oranges exist.
        # If this never reaches 0, the answer is -1.
        fresh = 0

        # Step 1: Scan the grid.
        # Add all rotten oranges to the BFS queue.
        # Count all fresh oranges.
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    fresh += 1
                elif grid[r][c] == 2:
                    queue.append((r, c))

        # If there are no fresh oranges initially, no time is needed.
        if fresh == 0:
            return 0

        # Four possible directions: down, up, right, left.
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        minutes = 0

        # Step 2: Multi-source BFS.
        # Each BFS level represents one minute.
        while queue and fresh > 0:
            # Number of rotten oranges that will spread rot during this minute.
            level_size = len(queue)

            # Process exactly the current level.
            # Newly rotten oranges are added to the queue,
            # but they will only spread rot in the next minute.
            for _ in range(level_size):
                r, c = queue.popleft()

                # Try to rot all 4-directionally adjacent fresh oranges.
                for dr, dc in directions:
                    nr = r + dr
                    nc = c + dc

                    # Check bounds and whether the neighbor is fresh.
                    if (
                        0 <= nr < rows
                        and 0 <= nc < cols
                        and grid[nr][nc] == 1
                    ):
                        # Rot this fresh orange.
                        grid[nr][nc] = 2

                        # One fewer fresh orange remains.
                        fresh -= 1

                        # This newly rotten orange can spread rot in future minutes.
                        queue.append((nr, nc))

            # After processing one full BFS level, one minute has passed.
            minutes += 1
```

```
# If all fresh oranges became rotten, return the elapsed time.
# Otherwise, some fresh oranges were unreachable.
return minutes if fresh == 0 else -1
```

4. Complexity Analysis

Let:

- m be the number of rows
- n be the number of columns

Time Complexity: $O(m * n)$

We scan the whole grid once to find fresh and rotten oranges.

During BFS, each orange cell is added to the queue at most once. For each cell, we check up to 4 neighbors.

Therefore, total time is:

```
O(m * n)
```

Space Complexity: $O(m * n)$

The queue can contain up to all cells in the worst case, for example if every cell is initially rotten.

Therefore, space complexity is:

```
O(m * n)
```

The grid is modified in-place, so we do not need an additional visited matrix.

5. Alternative Approaches

1. Repeated Simulation

You could repeatedly scan the whole grid minute by minute and rot oranges adjacent to currently rotten oranges.

However, this is less efficient because each minute requires scanning the full grid.

Worst-case time complexity could be:

```
O((m * n)^2)
```

2. DFS from Each Rotten Orange

You might try DFS from each rotten orange, but DFS does not naturally model simultaneous spreading by minutes. You would need extra distance tracking, and handling multiple sources becomes more complicated.

BFS is the natural fit because it processes nodes level by level, where each level corresponds exactly to one minute.

3. In-Place Timestamp BFS

Instead of using a separate `minutes` variable, you can store timestamps directly in the grid:

- Initial rotten oranges are 2
- Oranges rotten at minute 1 become 3
- Oranges rotten at minute 2 become 4

Then the answer is the largest value minus 2.

This is also BFS-based and has the same complexity.